Introduction to Low-Latency System with C++

Qishi C++ Study Group

Junshi Wang

February 22, 2026
8:00 PM EST

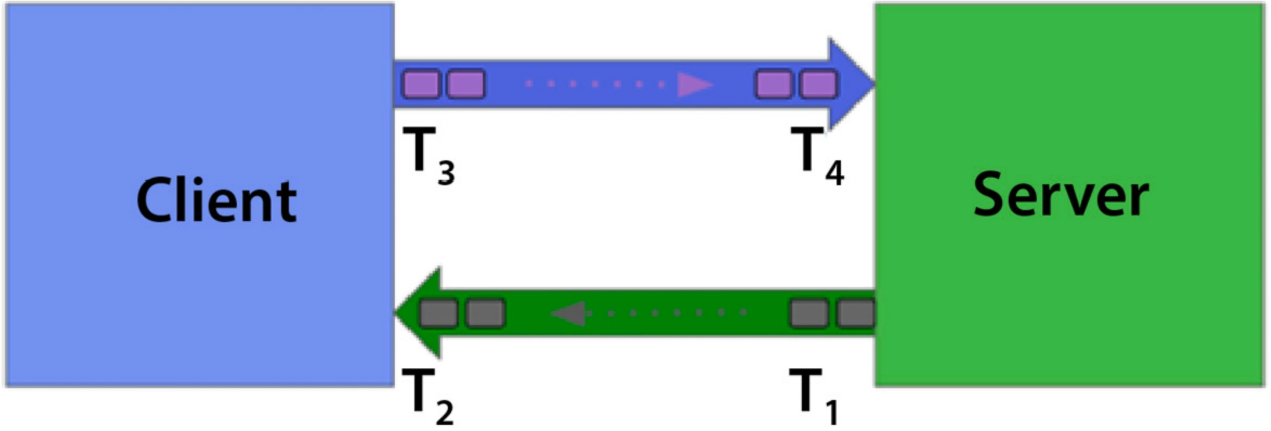
"Tomorrow's code is only as fast as today's design."

Agenda

1. What is latency, why is it important?
2. What is C++, why it is suitable for low latency?
3. Applications: Video Streaming, Gaming, IoT/Edge AI, Trading/Crypto
4. Next weeks and Focus

What is Latency?

- **Definition:** Time delay from when a task starts to when it completes
- **Low-latency systems focus:** Microsecond- or nanosecond-level response times
- **Metrics:**
  - RTT (Round-Trip Time)
  - TTT (Time-To-Target)
  - CPU clock cycles



Why Does Latency Matter?

Latency-Sensitive

- Better latency = Better experience/profit
- HFT, Gaming, Video Streaming

Latency-Critical

- Exceeding threshold = System Failure
- Autonomous Driving, Traffic Control

Impact

Microseconds of delay directly affect:

- Profitability
- User experience
- System correctness

Example: Electronic trading —being slower by 10μs = missing profitable opportunity or incurring losses

Key Design Implication

Low Latency $\neq$ Just "Fast Code"

Requirements:

- Correctness, Robustness, Throughput
- Average Latency, Peak Latency Cap
- Minimizing jitter (latency variance)

Leverage C++ low-level control:

- Optimize memory layout, CPU cache utilization, hardware interaction
- Avoid dynamic allocations on the critical path

What is C++?

Definition

Compiled, statically-typed, multi-paradigm programming language

C++ Core Capabilities

- **Low-level hardware access** (like C)
  - Direct memory management via pointers
- **High-level abstractions**
  - Classes, templates, generic programming
- **Zero-cost abstractions**
  - High-level features compile to efficient machine code without runtime overhead

Why C++ for Low-Latency?

Execution Model

- **Compiled Language**
  - No interpreter overhead
  - Ahead-of-time optimization
- **Deterministic Resource Usage**
  - Predictable memory allocation
  - No garbage collection pauses

Performance & Hardware

- **Close to Hardware**
  - Direct memory access
  - Cache-aware programming
- **Speed and High Performance**
  - Minimal runtime overhead
  - Fine-grained control

Language Constructs & Ecosystem

- **Portability**
  - Cross-platform support (Windows, Linux, macOS, Embedded)
- **Compiler Optimization**
  - Advanced optimizations (GCC, Clang, MSVC)
- **Type Safety**
  - Statically typed for compile-time safety
- **Multiple Paradigm**
  - OOP, functional, generic programming support
- Active language in development (C++17/20/23)

The System View of Latency (Framework)

End-to-End Perspective

Input (Camera/Controller) → Processing → Transmission → Output (Screen/Action)

Latency-Sensitive

- Better latency = Better experience/profit
- Example: Video Streaming

Latency-Critical

- Exceeding threshold = System Failure
- Example: Autonomous Driving, HFT

Key Metrics Beyond Average: Mean/Median, Peak Latency, Variance (Jitter)

Case Study 1: Live Video Streaming System - Pipeline

System Pipeline: Glass-to-Glass

1. Capture: Camera hardware latency
2. Encode: Transcoding/Transrating (CPU/GPU bound)
3. Transport: Network propagation, CDN hops, Protocol overhead
4. Buffer & Decode: Jitter buffer, Client-side decoding
5. Render: Display on viewer device

Key Constraint & Metric

Glass-to-Glass Latency

Video Streaming: Design & Optimization Logic

- **Protocol Selection:**
  - HTTP-based (HLS/DASH): Scalable, higher latency (segment buffering)
  - WebRTC (P2P/UDP): Low latency (<500ms), complex scaling
- **Architecture:** Edge Computing & CDN (Reduce physical distance)
- **Trade-off:** Accepting slight quality loss (compression) for lower transmission latency

Case Study 2: Gaming System - Pipeline & Metrics

System Pipeline: Input-to-Photon

1. Input: Peripheral polling (Mouse/Keyboard)
2. Logic: Game state update, Physics, Network Sync
3. Render: GPU draw calls, Frame composition
4. Display: Monitor refresh rate, Pixel response

Key Metrics

- Input Lag
- Ping (RTT)
- FPS
- Frame Time Consistency

Constraints

- Determinism (Lock-step simulation)
- Resource Contention (CPU/GPU vs. Network)

Gaming: Design & Optimization Logic

- **Network:** UDP vs. TCP, Client-side Prediction, Lag Compensation
- **Engine Optimization:**
  - Frustum Culling: Don't render what isn't seen
  - Memory Management: Avoid dynamic allocation in game loop (Memory Pools)
  - Task Prioritization: Input/Render > Background tasks
- **System Tuning:** Disable V-Sync, CPU Affinity (reduce context switches)

Application 3: IoT and Edge AI - Telco Layer

Current Technical Approaches for Reducing AI Inference Latency

- Model Optimization: quantization, pruning
- Caching: Prompt Caching + Semantic Caching
- Hardware Acceleration

Telco Network Infrastructure:

- Core Data Centers
- Regional (Metro) Data Centers
- Multi-Access Edge Computing (MEC) Nodes
- Near-RAN (Radio Access Network) Edges

References: Solving AI Foundational Model Latency with Telco Infrastructure, Semantic Caching for Large Language Models

Application 3: IoT and Edge AI - AI Inference Layer (vLLM)

What is vLLM?

Open-source LLM inference & serving framework built for high throughput + low latency
Architecture: Python frontend (API/scheduling) + C++/CUDA backend (memory/kernel execution)

vLLM Inference Lifecycle:

- Pre-processing: Tokenization, prompt encoding (CPU)
- KV Cache: PagedAttention block allocation (memory-bound)
- Kernel Execution: CUDA attention + FFN (GPU-bound)
- Post-processing: Sampling, decoding, streaming (mixed)

Main Contribution: Paged Attention(source code)

Applies OS virtual memory paging concepts to KV Cache management

vLLM: C++ Implementation Details

- **Better Memory Block Allocation with C++**
  - Fixed-size blocks + logical continuity via pointer arithmetic
  - Manual GPU memory control (cudaMalloc/cudaFree)
  - Memory alignment reduces GPU transaction overhead
- **C++ Deterministic Resource Control**
  - Pre-allocated memory pools (zero fragmentation)
  - Lock-free scheduler (std::atomic-based)
  - CPU affinity (pthread_setaffinity_np)
- **C++ + CUDA Deep Integration**
  - Kernel Fusion via C++ templates
  - PTX Inline Assembly for GPU instructions
  - Compile-Time Specialization (constexpr + templates)

Application 4: Trading and Crypto

Exchange Matching Engine

- Single Thread
- Memory Management
- Cache-line alignment

Market Maker Optimization

- Kernel Bypass (DPDK/Onload)
- FPGA Offload
- Huge Pages

On-chain Low Latency & MEV

- Lock free queue
- Priority queue
- Memory pool

Next Weeks and Focus

Core Building Blocks for Low-Latency Systems

- Threads and Concurrency Model
- Memory Pool Design
- Lock-Free Queue Implementation
- Low-Latency Logging
- TCP/UDP Utilities

Goal

Master the foundational components required for building deterministic low-latency systems.

Q&A / Discussion